Daffodil International University

LAB REPORT

Course Code: CSE114

Course Title: Programming and Problem Solving Laboratory

Experiment No: Mini Lab Project

Experiment Name: Distribution Management System

Submitted To:

Name: Md. Ashraful Islam Talukder

Designation: Lecturer

Department: Computer Science and Engineering

Daffodil International University.

Submitted By:

Name	ID
Maliha Tasnim	261-15-338
Anika Akter Mim	261-15-099
Touhidul Islam	261-15-377
Umma Amina	261-15-114
Md Abir Hossain	261-15-001

Section: —

Semester: Summer 2026

Department: Computer Science and Engineering

Daffodil International University.

Submission Date:

August 08, 2026

Declaration

We hereby declare that this lab project has been done by us under the supervision of **Md. Ashraful Islam Talukder**, Lecturer, Department of Computer Science and Engineering, Daffodil International University. We also declare that neither this project nor any part of this project has been submitted elsewhere as lab projects.

Submitted To

Md. Ashraful Islam Talukder
Lecturer
Department of Computer Science and Engineering
Daffodil International University
Signature: _____ *Date:* _____

Submitted By

Student Name: Maliha Tasnim **ID:** 261-15-338 Dept. of CSE, DIU
Signature: _____

Student Name: Anika Akter Mim **ID:** 261-15-099 Dept. of CSE, DIU
Signature: _____

Student Name: Touhidul Islam **ID:** 261-15-377 Dept. of CSE, DIU
Signature: _____

Student Name: Umma Amina **ID:** 261-15-114 Dept. of CSE, DIU
Signature: _____

Student Name: Md Abir Hossain **ID:** 261-15-001 Dept. of CSE, DIU
Signature: _____

Course & Program Outcome

The following course has course outcomes as follows:

CO	Statements
CO1	Learn a basic programming language (C/C++) to imitate and manipulate various problem solving techniques.
CO2	Demonstrate skills to analyze competitive programming problems and identify to choose appropriate language constructs and data structures for designing, building and testing realistic, complex applications.
CO3	Work collaboratively in teams to design, develop, and implement a comprehensive software project, demonstrating effective communication, coordination, and integration of team efforts to meet project goals.
CO4	Communicate complex engineering concepts effectively through reports, presentations, and instructions to both technical and non-technical audiences.

Table 1: Course Outcome Statements

CO	PO	Blooms	KP	CEP
CO1	PO1	C1, C2, P1, P2, A1, A2	K1, K2, K3, K4	EP1, EP2, EP3
CO2	PO2	C3, C4, P2, P3, A2, A3	K1, K2, K3, K4	EP1, EP2, EP3
CO3	PO9	C3, C4, C5, P3, P4, A3, A4		
CO4	PO10	C6, P2, A4		

Table 2: Mapping of CO, PO, Blooms, KP and CEP

The mapping justification of this table is provided in section 4.3.1, 4.3.2 and 4.3.3.

Table of Contents

Chapter 1

Introduction

This chapter introduces the Distribution Management System (DMS) project, detailing the background of the problem and the motivation behind developing a modular, terminal-based inventory application in C. It also outlines the project's core objectives, feasibility, gap analysis, and the expected outcomes for simulating a three-level product distribution chain.

1.1 Introduction

In the real world, most products do not travel directly from a manufacturer to a customer. Instead they pass through a distribution chain. A company produces goods and stores them in a warehouse. Dealers (also called distributors) buy these goods in bulk and store them regionally. Retailers, which are the local shops that customers actually visit, buy smaller quantities from a nearby dealer. Keeping track of how much stock exists at every level of this chain is a classic inventory management problem.

Traditionally, small businesses track this flow using paper ledgers or disconnected spreadsheets. Those methods are slow, error-prone, and hard to keep consistent when multiple people place orders. Commercial distribution systems solve the problem with large database-driven applications, but the same core logic — checking stock, transferring quantities, and recording the result — can be expressed with simple arrays and functions in C.

This project proposes the development of a **Distribution Management System** utilizing the C programming language. The system models a three-level chain (Company → Dealer → Retailer), provides role-based console panels, validates orders before stock moves, and organises the code into six communicating modules. Password-protected access is included for company, dealer, and retailer panels. By limiting the implementation to fundamental C concepts — variables, arrays, loops, conditionals, and functions — the project remains easy to read, explain, and extend.

1.2 Motivation

The computational motivation for this project is to replace informal, error-prone stock tracking with a structured digital simulation of a supply chain. Handling orders computationally ensures that stock is never created or destroyed by mistake: every successful transfer removes the ordered quantity from the seller and adds the same quantity to the buyer.

Solving this problem personally reinforces understanding of multi-file C programs, header-based declarations with **extern**, parallel arrays as a beginner-friendly data model, and defensive input validation. It is recommended that learners practice this kind of modular console application before moving on to database-backed inventory systems, because the underlying transfer rules remain the same.

1.3 Objectives

The primary objectives of developing this Distribution Management System are as follows:

1. To develop a terminal-based three-level distribution simulation (company, dealer, retailer) in C.
2. To implement role-specific panels that allow each user type to view the stock that belongs to them.
3. To allow dealers to order from the company and retailers to order from their linked dealer, with automatic stock transfer.
4. To reject invalid operations such as unknown IDs, non-positive quantities, and insufficient seller stock.
5. To organise the program into separate modules that communicate through header files.
6. To keep the entire codebase limited to fundamental C concepts: if-else, loops, arrays, and functions.

1.4 Feasibility Study

Existing commercial inventory and ERP systems are often resource-intensive, relying on complex web frameworks and heavy SQL databases. However, a review of similar console-based applications demonstrates that lightweight, offline simulations are highly feasible for teaching and for low-resource environments. Methodologically, foundational literature on C programming [1] emphasises clear modular structure and careful use of arrays for related data. This C-based project proves highly feasible by successfully implementing core inventory concepts — shared stock arrays, ID-based lookup, and guarded transfers — using only the standard C library and the GCC toolchain. It is recommended to utilise this lightweight architecture as a foundational, easily deployable model before introducing file persistence or a full database.

1.5 Gap Analysis

1. The absence of a simple educational tool that shows how stock moves through a multi-level distribution chain using only first-course C concepts.
2. The lack of modular multi-file examples that still remain easy for beginners to read, compile, and explain.

3. The inefficiency of informal paper or spreadsheet tracking for demonstrating order validation rules such as insufficient stock.
4. The gap between single-file beginner exercises and real software, where responsibilities are split across modules linked by headers.
5. The heavy resource dependence of commercial inventory systems, creating demand for a local, lightweight terminal-based teaching model.

1.6 Project Outcome

1. A fully functional, lightweight terminal-based application that simulates company, dealer, and retailer roles with password-protected panels.
2. Automatic and consistent stock transfer when dealers order from the company and retailers order from their assigned dealer.
3. Clear tabular console views for products, dealers, retailers, and stock levels at every stage of the chain.
4. Defensive validation that rejects unknown IDs, invalid quantities, and overselling.
5. A modular six-file C codebase that demonstrates header-based communication and parallel-array data modelling.

Chapter 2

Architecture

This chapter presents the architectural framework and operational flow of the Distribution Management System. It focuses on the design specifications of the modular C application, featuring flowcharts that illustrate user interactions and stock-transfer pathways, followed by the overall project plan.

2.1 Design Specification

The system is organised around six modules. Shared inventory data lives in **data.c** / **data.h**. Presentation helpers live in **display.c** / **display.h**. Role behaviour is implemented in **company.c**, **dealer.c**, and **retailer.c**. Execution begins in **main.c**.

Module	Files	Responsibility
Entry	main.c	Initialises data and routes users to role panels
Data	data.c / data.h	Global arrays, init_data(), product listing helpers
Display	display.c / display.h	Menus, tables, success/error messages, password check
Company	company.c / company.h	Company login and warehouse/dealer/product views
Dealer	dealer.c / dealer.h	Dealer login, stock view, order from company
Retailer	retailer.c / retailer.h	Retailer login, stock view, order from linked dealer

Table 2.1: Modules and responsibilities

Overall System Flow

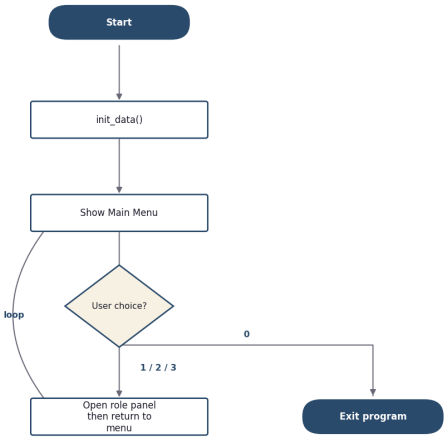

Fig 2.1: Main program control flow

Distribution Chain

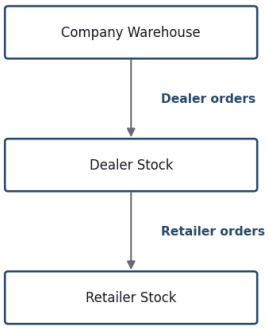

Fig 2.2: Product flow through the three-level chain

Password Verification

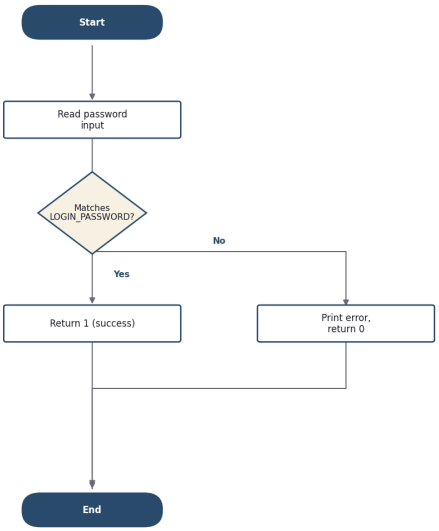

Fig 2.3: verify_password() function

Dealer Order From Company

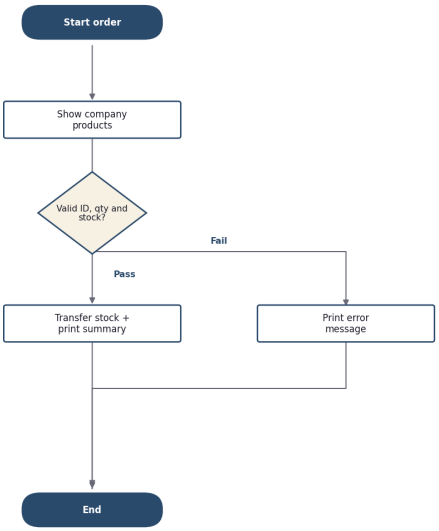

Fig 2.4: order_from_company() — three validation gates

Retailer Order From Dealer

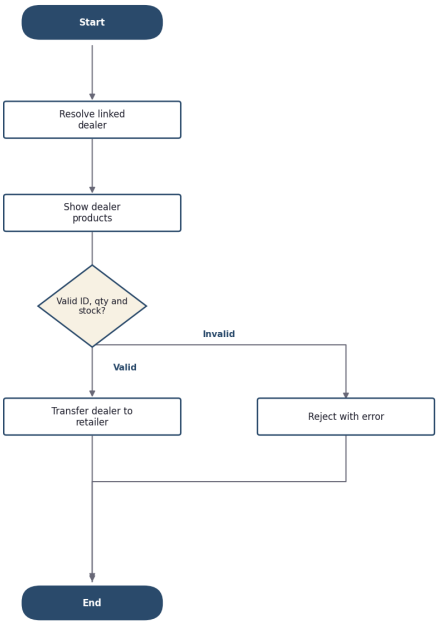

Fig 2.5: order_from_dealer() control flow

ID Lookup Pattern

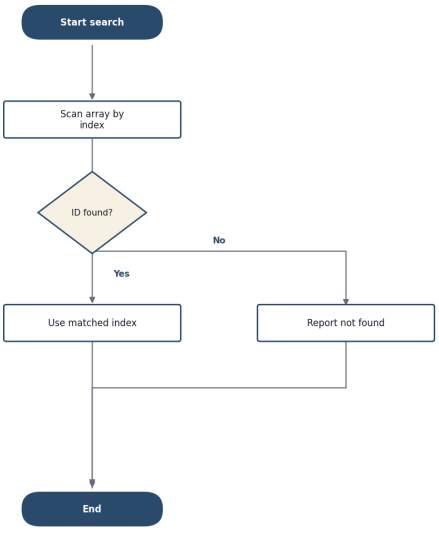

Fig 2.6: Linear ID-to-index search used across modules

> **Key architectural idea: stock arrays are defined once in data.c and declared with extern in data.h so every panel module can read and update the same shared inventory state.**

2.2 Overall Project Plan

The overall project plan adopts a systematic approach to develop the Distribution Management System. Initially, the shared data model and sample products/dealers/retailers were designed using parallel arrays. Next, display helpers and the three role panels were implemented so each actor could inspect stock. Finally, order-transfer logic with validation gates and password-protected login were integrated to ensure a consistent, beginner-friendly terminal application that preserves stock integrity across the chain.

Chapter 3

Implementation and Results

This chapter details the practical implementation of the C-based Distribution Management System, highlighting the programming constructs, modular organisation, and validation algorithms applied. It then presents the system's operational outputs with technical discussion of the underlying logic.

3.1 Implementation

3.1.1 Data Model (data.h)

Capacity limits and shared declarations live in the data header. Parallel arrays keep related fields aligned by index without using structures.

c

```c
#define MAX_PRODUCTS 10
#define MAX_DEALERS 5
#define MAX_RETAILERS 5

extern int  product_id[MAX_PRODUCTS];
extern char product_name[MAX_PRODUCTS][50];
extern int  company_stock[MAX_PRODUCTS];
extern int  product_total;

extern int  dealer_id[MAX_DEALERS];
extern char dealer_name[MAX_DEALERS][50];
extern int  dealer_stock[MAX_DEALERS][MAX_PRODUCTS];
extern int  dealer_total;

extern int  retailer_id[MAX_RETAILERS];
extern char retailer_name[MAX_RETAILERS][50];
extern int  retailer_dealer[MAX_RETAILERS];
extern int  retailer_stock[MAX_RETAILERS][MAX_PRODUCTS];
extern int  retailer_total;
```

3.1.2 Main Menu (main.c)

Execution starts in main(). It calls init_data() once, then uses a do-while loop and if-else routing to open the selected panel.

c

```
int main() {
    int choice;
    init_data();
    do {
        print_menu_header("DISTRIBUTION MANAGEMENT SYSTEM");
        print_menu_item(1, "Company Login");
        print_menu_item(2, "Dealer Login");
        print_menu_item(3, "Retailer Login");
        print_menu_item(0, "Exit");
        printf("\nEnter your choice: ");
        scanf("%d", &choice);
        if (choice == 1) company_menu();
        else if (choice == 2) dealer_menu();
        else if (choice == 3) retailer_menu();
        else if (choice != 0)
            print_error("Invalid choice. Please try again.");
    } while (choice != 0);
    return 0;
}
```

3.1.3 Password Gate (display.c)

All three role panels call verify_password() before granting access. The shared password is defined as a preprocessor constant.

c

```
#define LOGIN_PASSWORD "1122"

int verify_password() {
    char password[20];
    printf("Enter Password: ");
    scanf("%s", password);
    if (strcmp(password, LOGIN_PASSWORD) == 0) return 1;
    print_error("Invalid password.");
    return 0;
}
```

3.1.4 Stock Transfer (dealer.c)

Ordering is the heart of the system. Before any stock moves, the program runs three checks: the product must exist, the quantity must be positive, and the seller must have enough units.

c

```
if (found == 0) {
    print_error("Product not found. Check the table above.");
    return;
}
if (quantity <= 0) {
    print_error("Quantity must be greater than 0.");
    return;
}
if (company_stock[product_index] < quantity) {
    print_error("Not enough stock in company warehouse.");
    return;
}
company_stock[product_index] -= quantity;
dealer_stock[dealer_index][product_index] += quantity;
print_order_summary(product_name[product_index], quantity,
                    "Company", dealer_name[dealer_index]);
```

3.1.5 Retailer Link Constraint

Each retailer is permanently linked to exactly one dealer through retailer_dealer[]. Mirpur Shop buys only from Dhaka Dealer; Gulshan Shop buys only from Chittagong Dealer. The retailer order path resolves that link before showing available products.

3.1.6 Compilation

Because the program is spread across six .c files, all of them must be compiled and linked together:

BASH

```
gcc -o dms main.c data.c company.c dealer.c retailer.c display.c
./dms
```

Concept	Where it appears
Variables & arrays	All stock, ID, and name arrays in data.c
Loops	Listing tables; ID search; menu do-while
if-else	Menu routing; validation gates
Functions	One responsibility per helper/menu function
Header files	Declarations shared across modules
extern	data.h exposes arrays defined in data.c
2D arrays	dealer_stock / retailer_stock by owner × product

Table 3.1: Fundamental C concepts used in the project

3.2 Results and Discussion

In this section, the core operational outputs of the Distribution Management System are presented, along with technical discussions detailing the underlying C programming logic.

3.2.1 Main Menu

```
==============================================
  DISTRIBUTION MANAGEMENT SYSTEM
==============================================

  [1] Company Login
  [2] Dealer Login
  [3] Retailer Login
  [0] Exit
```

Discussion: The main menu is rendered by print_menu_header() and print_menu_item(). A do-while loop keeps the application alive until the user selects 0. Invalid numeric choices are rejected with print_error() without terminating the program.

3.2.2 Company Login and Panel

```
==============================================
  SUCCESS: Login successful.
==============================================

==============================================
  COMPANY PANEL
==============================================

  [1] View All Dealers
  [2] View All Products
  [3] View Company Stock
  [0] Back to Main Menu
```

Discussion: company_menu() first calls verify_password(). On success it enters a nested menu loop for viewing dealers, products, and warehouse stock. This isolates company read-only inspection from ordering, which is intentionally performed by dealers.

3.2.3 Viewing Dealers and Products

```
--- All Dealers ---

+------+----------------------+
| ID   | Dealer Name          |
+------+----------------------+
| 1    | Dhaka Dealer         |
| 2    | Chittagong Dealer    |
+------+----------------------+

--- All Products ---

+------+----------------------+------------+
| ID   | Product Name         | Stock      |
+------+----------------------+------------+
| 1    | Rice                 | 100        |
| 2    | Oil                  | 80         |
| 3    | Sugar                | 60         |
+------+----------------------+------------+
```

Discussion: list_dealers() and list_products() iterate from 0 to dealer_total / product_total and print rows through the display helpers. Using shared table printers keeps formatting consistent across panels.

3.2.4 Dealer Order From Company

```
--- Place Order to Company ---

--- Company Products (Available to Order) ---

+------+----------------------+------------+
| ID   | Product Name         | Stock      |
+------+----------------------+------------+
| 1    | Rice                 | 100        |
| 2    | Oil                  | 80         |
| 3    | Sugar                | 60         |
+------+----------------------+------------+

Enter Product ID: 1
Enter Quantity: 10

================================================
  ORDER PLACED SUCCESSFULLY
================================================
  Product  : Rice
  Quantity : 10 units
  From     : Company
  To       : Dhaka Dealer
================================================
```

Discussion: After selecting Product ID 1 (Rice) and quantity 10, order_from_company() subtracts 10 from company_stock and adds 10 to dealer_stock for Dhaka Dealer. The order

summary confirms product, quantity, source, and destination in one place.

3.2.5 Updated Dealer Stock

```
--- My Stock ---
Logged in as: Dhaka Dealer

+------+----------------------+------------+
| ID   | Product Name         | Stock      |
+------+----------------------+------------+
| 1    | Rice                 | 30         |
| 2    | Oil                  | 15         |
| 3    | Sugar                | 10         |
+------+----------------------+------------+
```

Discussion: Viewing stock after the order shows Rice rising from 20 to 30. This verifies the invariant that seller reduction equals buyer increase — no stock is created or destroyed by the transfer.

3.2.6 Retailer Order From Dealer

```
--- Place Order to Dealer ---
Your Dealer: Dhaka Dealer

--- Dealer Products (Available to Order) ---
Dealer: Dhaka Dealer

+------+----------------------+------------+
| ID   | Product Name         | Stock      |
+------+----------------------+------------+
| 1    | Rice                 | 30         |
| 2    | Oil                  | 15         |
| 3    | Sugar                | 10         |
+------+----------------------+------------+

Enter Product ID: 2
Enter Quantity: 3

================================================
  ORDER PLACED SUCCESSFULLY
================================================
  Product  : Oil
  Quantity : 3 units
  From     : Dhaka Dealer
  To       : Mirpur Shop
================================================
```

Discussion: Mirpur Shop is linked to Dhaka Dealer via retailer_dealer[0] = 1. The retailer can only order from that dealer. Ordering 3 units of Oil reduces dealer stock and increases retailer stock by the same amount.

3.2.7 Insufficient Stock Rejection

```
Enter Product ID: 1
Enter Quantity: 500

  ERROR: Not enough stock in company warehouse.
```

Discussion: When the requested quantity exceeds company_stock, the transfer is aborted before any array is modified. Early returns after each failed check prevent partial updates and keep inventory consistent.

Test Case	Input	Expected Result	Observed
Valid company login	Password 1122	Open company panel	Pass
Invalid password	Wrong string	Error, return to menu	Pass
Dealer orders Rice ×10	ID 1, qty 10	Company 100→90, Dealer 20→30	Pass
Oversell attempt	qty 500	Error, no stock change	Pass
Unknown product ID	ID 99	Product not found	Pass
Retailer orders Oil ×3	ID 2, qty 3	Dealer↓, Retailer↑ by 3	Pass
Invalid menu choice	choice 9	Error, menu repeats	Pass

Table 3.2: Manual test cases and results

> **The most important invariant tested: after any order, the amount removed from the seller always equals the amount added to the buyer.**

Chapter 4

Engineering Standards and Mapping

This chapter explores the engineering standards applied in developing the Distribution Management System. It also evaluates the project's broader impacts on society, environment, ethics, and its long-term sustainability.

4.1 Impact on Society, Environment and Sustainability

Implementing a digital distribution simulation modernises how learners and small operators reason about inventory flow. By replacing informal paper tallies with consistent computational checks, it improves operational clarity, reduces wasteful overselling mistakes, and promotes sustainable digital practices.

4.1.1 Impact on Life

The system improves day-to-day understanding for students, instructors, and small business operators who need a clear mental model of company–dealer–retailer stock movement. Automating validation reduces stress around manual arithmetic mistakes, and role-based panels make responsibilities easier to learn and demonstrate.

4.1.2 Impact on Society & Environment

Societally, the project promotes digital literacy in supply-chain thinking using accessible tools (GCC and a terminal). Environmentally, practising inventory logic in software rather than on repeated paper worksheets supports a paperless learning workflow and reduces printed materials during labs and demonstrations.

4.1.3 Ethical Aspects

Ethical considerations include fair access control and honest reporting of stock. Password verification limits casual misuse of role panels during demonstrations. The transfer logic treats all products and partners by the same rules, avoiding biased handling. Students are encouraged to treat sample business data carefully when extending the system toward real deployments.

4.1.4 Sustainability Plan

The software is built for long-term classroom sustainability: six small modules, no third-party libraries, and constants that make capacity easy to retune. Future sustainability efforts include adding file I/O for persistence, pricing/payment fields, runtime registration of partners, and eventually migrating from in-memory arrays to a relational store while keeping

the same transfer rules.

4.2 Project Management and Team Work

This project was executed as a group effort, where tasks were divided according to the individual strengths and capabilities of each team member. Programming, documentation, testing, sample-data preparation, and demonstration planning were coordinated so the Distribution Management System could be delivered as a complete lab project.

4.2.1 Team Member Contributions

- **Md Abir Hossain (ID: 261-15-001) — Lead Developer:** Designed the modular architecture and implemented the core C modules (data, display, company, dealer, retailer), including order validation and stock-transfer logic.
- **Maliha Tasnim (ID: 261-15-338) — Documentation Lead:** Organised the engineering report structure, wrote chapter narratives to match the CSE114 lab format, and ensured consistency of figures, tables, and references.
- **Anika Akter Mim (ID: 261-15-099) — Testing & Quality:** Prepared manual test cases for valid and invalid orders, verified stock invariants after transfers, and recorded sample console sessions for the Results section.
- **Touhidul Islam (ID: 261-15-377) — Design & Flow:** Contributed to menu/UX flow design, flowchart specifications for architecture diagrams, and clarity of role-based panel interactions.
- **Umma Amina (ID: 261-15-114) — Data & Demo Preparation:** Helped define sample products, dealers, and retailers; prepared demonstration scripts for the live presentation before the course instructor.

4.2.2 Cost Analysis and Budget Required

Since this is a console-based program developed with open-source tools (GCC Compiler, VS Code / Cursor), the direct software licensing cost is zero. A hypothetical commercial budget for packaging a similar teaching or small-business tool includes:

- Development Cost (Hypothetical Developer Hours): 12,000 BDT
- Hardware Setup (Local desktop for operations): 40,000 BDT
- Initial Training & Deployment: 5,000 BDT
- **Total Primary Budget: 57,000 BDT**

Alternate Budget: Migrating from in-memory arrays to a cloud relational database for multi-user remote access would add approximately 1,000 BDT/month (~12,000 BDT/year). The primary budget is suitable for a single-machine classroom or shop demo; the alternate budget justifies recurring cost with durability and remote access.

4.2.3 Revenue Model

If commercialised as a teaching or micro-business toolkit, the software can follow a B2B one-time licensing model:

- **License Fee:** Institutions or small distributors purchase the standalone package for a fixed fee of 10,000–15,000 BDT per site.
- **Maintenance Subscription:** Optional 2,000–3,000 BDT/year for support, sample-data updates, and feature patches.

4.3 Complex Engineering Problem

4.3.1 Mapping of Program Outcome

In this section, a mapping of the problem and provided solution with targeted Program Outcomes (POs) is provided.

PO	Justification
PO1	CO1 covers PO1 by applying fundamental engineering knowledge and C programming concepts. Loops, conditionals, arrays, and functions (e.g., order_from_company, verify_password, init_data) build the foundational structure of DMS.
PO2	CO2 covers PO2 by analysing distribution requirements and selecting appropriate constructs: parallel arrays, 2D stock matrices, multi-file modular design, and linear ID search with validation gates for realistic order handling.
PO9	CO3 covers PO9 through organised project execution — separating architecture, implementation, testing, and documentation — demonstrating coordination of engineering activities toward a complete deliverable.
PO10	CO4 covers PO10 by communicating complex inventory logic through a clear console UI, structured flowcharts, and this comprehensive engineering report suitable for both technical and non-technical audiences.

Table 4.1: Justification of Program Outcomes

4.3.2 Complex Problem Solving

- CO1 is linked to EP1 as developing DMS requires in-depth use of basic C constructs (loops, conditionals, functions). EP2 is relevant because the design balances simplicity for beginners with realistic validation rules for stock integrity.
- CO2 is linked to EP1 because multi-file modular design with shared extern data and 2D stock arrays requires first-principles understanding beyond single-file exercises. EP2 also applies when balancing immediate in-memory simplicity against future persistence and scalability needs.

Attribute	Description	Mapped
EP1	Depth of Knowledge	Yes
EP2	Range of Conflicting Requirements	Yes
EP3	Depth of Analysis	—
EP4	Familiarity of Issues	—
EP5	Extent of Applicable Codes	—
EP6	Extent of Stakeholder Involvement	—
EP7	Inter-dependence	—

Table 4.2: Mapping with complex problem solving

4.3.3 Engineering Activities

Mapping with EA1 (Range of resources): Development required selecting open-source tools (GCC, a code editor) and standard C libraries (stdio.h, string.h). Inventory state uses process memory resources via global arrays, while console I/O resources present tables and collect orders. Modular headers manage the interaction surface between components.

Activity	Description	Mapped
EA1	Range of resources	Yes
EA2	Level of Interaction	—
EA3	Innovation	—
EA4	Consequences for society and environment	—
EA5	Familiarity	—

Table 4.3: Mapping with engineering activities

Chapter 5

Conclusion

This chapter concludes the project by summarising the key features and technical achievements of the Distribution Management System. It also evaluates current limitations and outlines potential future enhancements.

5.1 Summary

The Distribution Management System was successfully developed as a console-based application in C. It models a realistic three-level supply chain in which stock flows from company to dealer to retailer under role-specific panels. The project integrates modular multi-file organisation, parallel-array data modelling, password-protected access, tabular console views, and guarded order transfers that preserve inventory integrity. Overall, DMS is a practical demonstration of fundamental programming constructs applied to a recognizable business workflow.

5.2 Limitation

1. Lack of Graphical Interface: The system relies entirely on a command-line console and has no GUI.
2. No Persistent Storage: All stock lives in memory; changes are lost when the program exits.
3. Fixed Catalogue: Products, dealers, and retailers are hardcoded in init_data() and cannot be added at runtime.
4. No Pricing or Payments: Orders move quantities only; there is no cost calculation.
5. Basic Input Handling: Non-numeric input to scanf can confuse menus because inputs are read as plain integers.
6. Shared Password: All roles currently share one LOGIN_PASSWORD rather than per-user credentials.

5.3 Future Work

1. File I/O Persistence: Save and load stock with fopen/fprintf/fscanf so state survives between runs.
2. Pricing Module: Add product prices and compute the total cost of each order.
3. Runtime Registration: Allow adding products, dealers, and retailers while the program runs.

4. Order History Log: Keep a simple transaction history for review and auditing.
5. Low-Stock Warnings: Alert users when a product falls below a configurable threshold.
6. GUI or Web Front-End: Upgrade usability while keeping the same transfer rules in the core logic.

References

[1] Kernighan, B. W., & Ritchie, D. M. (1988). The C Programming Language (2nd ed.). Prentice Hall.

[2] GNU Project. (2024). GCC, the GNU Compiler Collection — Online Documentation. https://gcc.gnu.org/onlinedocs/

[3] TutorialsPoint. (2024). C Programming Tutorial — Multi-file Programs and Header Files. https://www.tutorialspoint.com/cprogramming/

[4] Chopra, S., & Meindl, P. (2019). Supply Chain Management: Strategy, Planning, and Operation (7th ed.). Pearson.

[5] Daffodil International University, Department of CSE. CSE114: Programming and Problem Solving Laboratory — Mini Lab Project Report Guidelines.